

eman ta zabal zazu



Universidad
del País Vasco

Euskal Herriko
Unibertsitatea

Space Invaders

Egileak:

Asier Ferandez, Ibai Gonzalez, Lander Villa eta Oier Zabalegi

Gaien Aurkibidea

1	Sarrera	2
1.1	Proiektuaren aurkezpena	2
1.2	Helburuak	2
2	Antolaketa	2
2.1	Lanaren antolaketa	2
2.2	Bilerak	2
2.3	Baliabideak	2
3	Diseinua	4
3.1	KD, arrazoitu erabakiak	4
3.2	MVC-ren azalpena	6
3.3	Patroien azalpena	7
3.3.1	Erabilitako Patroiak	7
3.4	Java8-ren azalpena	10
4	Garapena	10
4.1	Sprint bakoitzeko helburuak eta erabakiak	10
4.2	Atazen dokumentazioa	12
4.3	Proben dokumentazioa	14
5	Ondorioak	15
5.1	Hausnarketa	15
5.2	Etorkizuneko lanak	15

1 Sarrera

1.1 Proiektuaren aurkezpena

Dokumentu honetan aurkezten den proiektua Space Invaders joko klasikoaren bertsio baten garapena da. Joko honetan, jokalaria espaziontzi bat kontrolatzen du eta pantailaren goialdetik jaisten diren etsaiak suntsitu behar ditu, jokariaren espaziontzia ukitu edo pantailaren beheko aldera iritsi aurretik. Proiektu honetako helburu nagusiak diseinu patroiak eta softwarearen garapen metodologiak praktikan jartzea izan dira.

1.2 Helburuak

Proiektu honen helburu nagusiak honako hauek dira:

- Objektuei Orientatutako Programazioaren kontzeptu aurreratuak aplikatzea.
- Model-View-Controller arkitektura erabiliz aplikazioa egituratzea.
- Diseinu patroiak (Singleton, Factory, Observer, etab.) identifikatzea eta egoki inplementatzea.
- Bertsio-kudeaketa eta talde-lana bultzatzea.

2 Antolaketa

2.1 Lanaren antolaketa

Lana Sprint desberdinetan banatu da, eta bakoitzean funtzionalitate zehatz batzuk garatu dira. Taldekide bakoitzak zeregin espezifikoak hartu ditu bere gain, eta kodea integratzeko Git bidezko commit eta pull request-ak erabili ditugu.

2.2 Bilerak

Proiektuaren jarraipena egiteko, aldian behin bilerak egin ditugu. Bilera hauetan Sprint-aren egoera aztertu dugu, sortutako zailtasunak partekatu ditugu eta hurrengo pausoak definitu ditugu talde-lana arintzeko.

2.3 Baliabideak

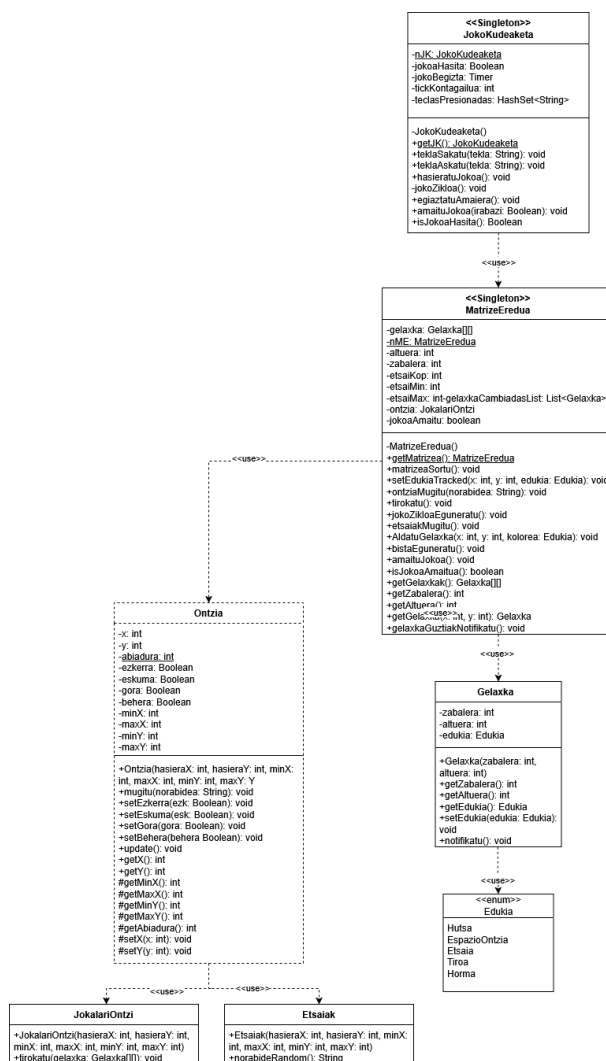
Garapena aurrera eramateko hainbat tresna erabili ditugu:

- **IDE:** Eclipse edo IntelliJ IDEA erabili dugu Java kodea idazteko.
- **Bertsioen kudeaketa:** Git eta GitHub erabili ditugu biltegia kudeatzeko eta taldekideen arteko kodea bateratzeko.
- **Komunikazioa eta Bilerak:** Discord erabili dugu bilerak egiteko eta eguneroko zalantzak argitzeko.
- **Dokumentazioa:** LaTeX erabili dugu azken txosten hau idazteko.

- **Eredu Generatiboak:** Proiektuan zehar adimen artifizialeko eredu generatiboak erabili ditugu (Claude, Gemini). Tresna hauek arazo zehatzak konpontzeko laguntza orokor gisa erabili dira, zehazki ondorengo ataletan:
 - **Harien kudeaketa eta Swing:** Swing liburutegianerabiltzaile interfazea sinkronizatzeko zailtasunak izan genituenean, `SwingUtilities.invokeLater` nola erabiltzeko, eta elementuen zerrendak asinkronoki aldatzeak sortutako `ConcurrentModificationException` erroreak nola konpondu erakutsi ziguten.
 - **Diseinu Patroiak:** MVC arkitektura ondo finkatzeko, eta bereziki Java-ren `Observer` eta `Observable` estandarrak gure `GelaxkaBista` eta `MatrizeEredua` klaseen artean nola inplementatu irakatsi ziguten. Era berean, **Factory** eta **Composite** patroiak (`EtsaiFactory` eta ontzien sorkuntza) egituratzeko adibide espezifikoak eskatu genizkien.
 - **Erroreen bilaketa (Debugging):** Jokoaren exekuzioan gertatutako salbuespenak, irudiak kargatzean eukitako arazoak.

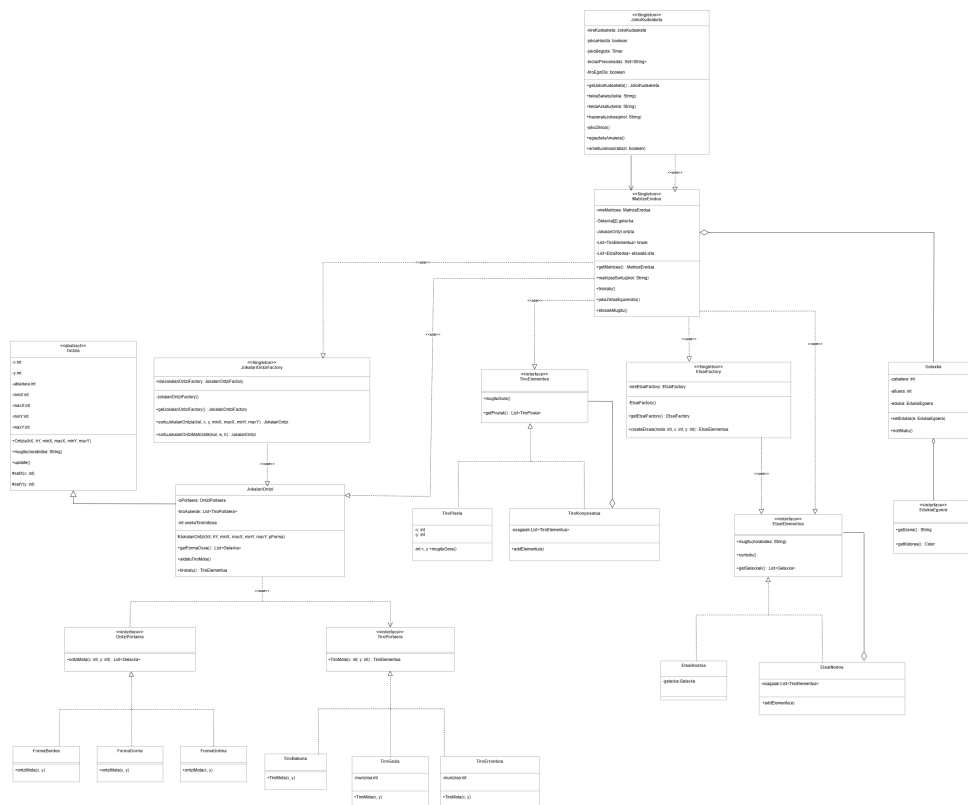
3 Diseinua

3.1 KD, arrazoitu erabakiak



Irudia 1: 1. Sprint-eko Klase Diagrama

1. Sprint-ean matrizea sortzeko eta jokoaren oinarritzko egitura finkatzeko erabakiak hartu genituen. Objektu guztiak pixel bakarrekoak zirenez, oinarritzko Eredua, Bista eta Kontrolatzaile klaseak sortu genituen, aplikazioaren arkitektura zimentatuz.



Irudia 2: 2. Sprint-eko Klase Diagrama

2. Sprint-ean pixel anitzeko espaziontziak eta etsaiak gehitu ditugunez, diseinua konplexuagoa bihurtu da. Horretarako, diseinu patroiak (Factory, etsaiak eta jokalaria modu errazagoan sortzeko) txertatu genituen, aurreko egitura moldatuz eta hedatuz klaseen arteko akoplamendua txikitzeko.



Irudia 3: 3. Sprint-eko Klase Diagrama

3. Sprint-ean azken hedapenak gehitu genituen, JAVA8-n hainbat funtzionalitate implementatuz eta diseinua behin betiko finkatuz. Hemen aurretik definitutako klaseen herentzia eta interfaze berriak gehitu genituen jokoari konplexutasuna emateko.

3.2 MVC-ren azalpena

Aplikazioa Model-View-Controller arkitektura patroia erabiliz diseinatu dugu. Honela, jokoaren logika (Eredua), erabiltzaile-interfazea (Bista) eta hauen arteko komunikazioa (Kontrolatzailea) banatu ditugu.

Ereduak datuak eta jokoaren egoera kudeatzen ditu. Adibidez, `Gelaxka` klasea `java.util.Observable` klasetik hedatzen da. Bere egoera aldatzen denean, bere behatzaileei (Observer) abisatzen die:

```
1 public void notifikatu() {
2     setChanged();
3     notifyObservers(this.edukia);
4 }
```

Listing 1: Gelaxka.java fitxategiko notifikatu metodoa

Bistan, `GelaxkaBista` klaseak `java.util.Observer` implementatzen du. Ereduak abisu bat bidaltzen duenean, `Bista` `update` metodoa exekutatzen du bere kolorea aldatzeko, eta bistaratzeko ardura du:

```
1 public class GelaxkaBista extends JLabel implements Observer {
2     // ...
3     @Override
4     public void update(Observable o, Object arg) {
5         if (arg instanceof Eredua.EdukiaEgoera) {
6             Eredua.EdukiaEgoera egoera = (Eredua.EdukiaEgoera) arg;
```

```

7         final Color bg = egoera.getKolorea();
8
9         SwingUtilities.invokeLater(() -> setKolorea(bg));
10    }
11 }
12 }

```

Listing 2: GelaxkaBista.java fitxategiko update metodoa

Lotura hori egiteko, JokoPanela klasean gelaxkak hasieratzen direnean, bista bakoitza bere ereduari gelaxkatik observer gisa gehitzen zaio:

```

1 GelaxkaBista g = new GelaxkaBista();
2 this.bistaMatrizea[x][y] = g;
3 matrizePanela.add(g);
4 Eredua.MatrizeEredua.getMatrizea().getGelaxka(x, y).addObserver(g);

```

Listing 3: JokoPanela.java fitxategian Observer-ak gehitzea

Ereduaren eta Bistaren arteko fluxua **Kontrolatzaileak** (JokoKudeaketa klaseak) gidatzen du. Adibidez, jokalaria tiro egiteko tekla sakatzen duenean, Kontrolatzaileak Ereduari (Matrizeari) agintzen dio logika prozesatzeko, Bista zuzenean ukitu gabe:

```

1 public void teklaSakatu(String tekla) {
2     if (tekla.equals("TIROA")) {
3         if (!tiroEginDa) {
4             teclasPresionadas.add(tekla);
5             tiroEginDa = true;
6         }
7     }
8 }
9
10 // Joko zikloaren barruan (timer bidez) kontrolatzaileak ereduari
11 // deitzen dio:
12 if (teclasPresionadas.contains("TIROA")) {
13     MatrizeEredua.getMatrizea().tirokatu();
14     teclasPresionadas.remove("TIROA");
15 }

```

Listing 4: JokoKudeaketa.java: Kontrolatzaileak Ereduarekin elkarreragiten

3.3 Patroien azalpena

MVC-az gain, beste diseinu patroia batzuk ere erabili ditugu, adibide nabarmen bat **Singleton** patroia delarik. Patroi honek bermatzen du klase baten instantzia bakarra sortzen dela aplikazio osoan zehar, eta horretarako sarbide global bat eskaintzen du.

3.3.1 Erabilitako Patroiak

- Singleton: Instantzia Bakarra Jokoa badiha klase batzuk "nagusiak" direnak eta joko osoan kopia bakarra egon behar dela ziurtatu behar dugu.
 - Non agertzen da: JokoKudeatzailea, MatrizeEredua eta fabrikatan.
 - Zertarako: Adibidez, jokoa matrizea bakarra izan behar da. Patroi honekin, jokoa edozein lekutatik kopia berera jotzen dugula ziurtatzen dugu datuak ez nahasteko.


```

1 public class MatrizEredua {
2     private static MatrizEredua nireMatrizea = new
    MatrizEredua();
3
4     private MatrizEredua() {
5         // Matrizearen hasieraketa
6     }
7
8     public static MatrizEredua getMatrizea() {
9         return nireMatrizea;
10    }
11 }

```

Listing 5: Singleton patroia MatrizEredua klasean

- Factory: Objektuak Sortzen objektu asko (inbaditzaileak, tiroak...) sortu behar direnean, hobe da horretarako leku bakar bat izatea.
 - Non agertzen da: JokoElementuFactory eta EtsaiFactory.
 - Zertarako: Jokoak fabrikari eskatzen dio objektu bakoitza eta hark ematen dio eginda. Horrela, etsai berri bat sartu nahi badugu, fabrikako kodea bakarrik aldatu behar dugu.

```

1 public EtsaiElementua createEtsaia(int mota, int x, int y) {
2
3     switch (mota) {
4         case 0: portaera = new PortaeraNormala(); break;
5         case 1: portaera = new PortaeraAzkar(); break;
6         case 2: portaera = new PortaeraTiratzaille(); break;
7         case 3: portaera = new PortaeraNormala(); break;
8         default: portaera = new PortaeraNormala(); break;
9     }

```

Listing 6: Factory patroia EtsaiElementua klasean

- Observer: Komunikazioa Errazten Ereduren eta Ikuspegiaren arteko independen-
tzia bermatzeko, Observer patroia funtsezkoa da.
 - Non agertzen da: MatrizEredua (Subject) eta (Observer).
 - Zertarako: Pantailak jakin behar du noiz mugitu den ontzi bat marrazkia eguneratzeko. Ereduek mezu bat bidaltzen die behatzaileei "zerbait aldatu da!" esateko, eta pantailak orduan bere lana egiten du.

```

1 public void notifikatu() {
2     setChanged();
3     notifyObservers(...);
4 }

```

Listing 7: Observer patroia

```

1 public void update(Observable o, Object arg) {
2     if (... .equals(arg)) {
3         //...
4
5     } else if (... .equals(arg)) {
6         //...
7     }
8 }

```

- Composite Patroia: Elementuen Hierarkia Patroi honen bidez, jokoaren elementu guztiak (jokalaria, etsaiak eta tiroak) modu berean tratatzea lortzen dugu, objektu bakarra izan ala multzo baten parte izan.

- Non agertzen da: JokoElementua interfazean eta haren azpian dauden TiroKonposatua, EtsaiNodua eta jokalaria kudeaketan.
- Zertarako: Sistemak ez du bereizten banakako elementu bat edo talde bat mugitzen ari den; denak "elementu" gisa ikusten ditu. Horrek logika asko errazten du, multzo osoak pieza bakar bat balira bezala kudeatu daitezkeelako.

```

1  public class EtsaiHostoa implements EtsaiElementua {
2      private Gelaxka gelaxka;
3
4      public EtsaiHostoa(int x, int y) {
5          this.gelaxka = new Gelaxka(x, y);
6      }

```

Listing 8: Composite patroia EtsaiHostoa klasean

- Strategy: Portaera Aldakorrak Objektu baten portaera (adibidez, nola egiten duen tiro) aldatu nahi badugu, "estrategia" hori aldatzen diogu.

- Non agertzen da: OntziPortaera eta TiroPortaera interfazeetan.
- Zertarako: Ontzi bati tiro mota aldatzeko, bere barruan duen "tiro egiteko modua" aldatzen diogu. Oso eroso da jokoan aukera ezberdinak sortzeko kodea gehiegi konplikatu gabe.

```

1  public interface TiroPortaera {
2      public TiroElementua TiroMota(int x, int y);
3  }

```

Listing 9: Strategu patroia TiroPortaera klasean

```

1  public class TiroBakuna implements TiroPortaera {
2      public TiroElementua TiroMota(int x, int y) {
3          return new TiroPixela(x, y);
4      }

```

- State : Gelaxken portaera Gelaxka bakoitzak une bakoitzean duen egoeraren arabera jokatzeko duela ziurtatzeko erabiltzen da, EdukiaEgoera egiturari esker.

- Non agertzen da: Gelaxka klasean eta, bereziki, EdukiaEgoera interfazean (eta honen egoera ezberdinetan: hustua, okupatua, babes-zatia...).
- Zertarako: Gelaxka bati tiro batek jotzen dionean, ez dugu if-else konplexurik behar. Gelaxkak bere egoerari (EdukiaEgoera) galdetzen dio zer egin (suntsitu, soinua egin edo ezer ez). Horrela, egoera berriak gehitzea oso erraza da eta kodea askoz garbiagoa geratzen da.

```

1  public interface EdukiaEgoera {
2      public String getIzena();
3      public abstract Color getKolorea();
4  }

```

Listing 10: State patroia EdukiEgoera klasean

```

1 public class EtsaiEgoera implements EdukiaEgoera {
2     @Override
3     public String getIzena() {
4         return "Etsaia";
5     }
6
7     @Override
8     public Color getKolorea() {
9         return Color.ORANGE;
10 }

```

3.4 Java8-ren azalpena

Proiektuaren azken Sprint-ean Java 8 bertsioak zekarzkien berrikuntzak (bereziki *Lambda* adierazpenak) txertatu ditugu kodean, honela garbitasuna eta irakurgarritasuna nabarmen hobetuz.

Adibidez, iteratzaileen edo begizten ordez `forEach` erabiliz zerrendak modu funtzionalen zeharkatu ditugu `TiroKonposatua.java` eta beste hainbat klasetan:

```

1 osagaiak.forEach(TiroElementua::mugituGora);
2 osagaiak.forEach(e -> l.addAll(e.getPixelak()));

```

Listing 11: Lambda adierazpenen erabilera `forEach` bidez

Bistan ere, exekuzio-harien kudeaketa egiteko `SwingUtilities.invokeLater` erabiltzean *Lambda* adierazpenak erabili ditugu, kode askoz laburragoa eta zuzenagoa lortuz objektu anonimoak erabili beharrean:

```

1 SwingUtilities.invokeLater(() -> setKolorea(bg));

```

Listing 12: `SwingUtilities` Lambda adierazpenarekin

Horrek Java 8aren funtzionalitate berrien eta Java estandarraren klase zaharragoen arteko oreka argi erakusten du.

4 Garapena

4.1 Sprint bakoitzeko helburuak eta erabakiak

1. Sprint-a

- **EI1: Espazio-matrizea hasieratu.** Jokoaren oinarritzko sareta sortu zen.
- **EI2: Pixel bakarreko espaziontzia inplementatu.** Ontziaren mugimendua eta pixel bakarreko tiroa inplementatu genituen lehen funtzionalitate gisa.
- **EI3: Pixel bakarreko etsaiak inplementatu.** Etsaien mugimendu basikoa zehaztu zen.
- **EI4: Hasierako panela sortu.** Kargatzeko pantaila sortu zen.
- **EI5: MVC aplikatu.** Arkitektura banaketa egin genuen (Eredua, Bista eta Kontrolatzailea).

2. Sprint-a

- **EI6: Hasierako panela aldatu, jokoa konfiguratze aukerak gehitu.** Hiru espaziontziren artean aukeratzeko ahalmena gehitu genuen: Green, Blue eta Red.
- **EI7: Pixel anitzeko espaziontziak eta etsaiak implementatu.** Orain elementu bakoitzak gelaxka bat baino gehiago hartzen ditu.
- **EI8: Espaziontzien tiro motak eta beren aldaketak implementatu.** Era askotako tiroak jaurtitzeko gaitasuna sartu zen.
- **EI9: Diseinu patroien implementazioa.** Hasierako diseinua hobetu eta formalizatu genuen klase berriak gehituz Factory edo patroien estrukturaleri bide emateko.

3. Sprint-a

- **EI10: Java8 gehitu implementazioan.** *Lambdas* erabili ziren kodea eta harien kudeaketa garbitzeko.
- **EI113: Hedapena implementatu.** Aukera posible hauen artean funtzionalitate berri hauek implementatu ziren erabaki bezala:
 - Puntuazioa eta jokalarien bizitzaren informazioa gehitu hasierako pantailan.
 - Etsai azkarrak sortu dira, etsaiek ere tiroekin erasotzeko aukera izanez, eta etsaien sorkuntza konplexuagoa eginez.
 - Jokoaren zailtasun motak eta partida mugagabea modua.

4.2 Atazen dokumentazioa

1. Sprint-eko Atazak

Sprint	Ataza	Arduraduna	Denbora planif.	Denbora erreal	Oharrak
1	ontziaMugitu()	Lander V	30 min	20 min	
1	EtsaiakMugitu()	Oier Z, Ibai G, Asier F, Lander V	2 ordu	3,5 ordu	Klasean irakasleari erakutsi ondoren, ia guztiz aldatu behar izan genuen
1	tirokatu()	Oier Z, Lander V	1 ordu	1,25 ordu	
1	matrizeaSortu()	Oier Z, Ibai G	1,5 ordu	2 ordu	MVC patroia berregin behar izan genuen
1	jokoZikloaEguneratu()	Asier F, Oier Z, Ibai G	2 ordu	1,5 ordu	MVC patroia berregin behar izan genuen
1	egiaztatuaAmaiera()	Asier F, Oier Z	30 min	20 min	
1	PreparatuMatrizea()	Asier F, Ibai G, Lander V, Oier Z	1 ordu	1,5 ordu	MVC patroia berregin behar izan genuen
1	hasieratu()	Oier Z, Ibai G, Asier F	45 min	1 ordu	
1	hasieraPanelaSortu()	Ibai G, Asier F	35 min	50 min	
1	LehioNagusia Klasea	Oier Z, Lander V, Asier F, Ibai G	4 ordu	3 ordu	MVC patroia berregin behar izan genuen
1	GelaxkaBista Klasea	Oier Z, Asier F, Ibai G	1 ordu	45 min	MVC patroia berregin behar izan genuen
1	JokoPanela klasea	Oier Z, Lander V, Asier F, Ibai G	1,5 ordu	1 ordu	
1	GameOverPantaila klasea	Asier F	20 minutu	25 minutu	
1	JokoKudeaketa Klasea	Oier Z, Lander V, Asier F, Ibai G	4 ordu	5,5 ordu	MVC patroia berregin behar izan genuen
1	JokalariOntzi Klasea	Ibai G, Asier F	15 minutu	20 minutu	
1	Edukia instantzia	Ibai G	1 minutu	1 minutu	
1	Etsaiak klasea	Ibai G, Asier F	20 minutu	20 minutu	
1	Gelaxka Klasea	Oier Z, Lander V, Asier F, Ibai G	20 minutu	20 minutu	
1	Main klasea	Ibai G, Oier Z	10 minutu	15 minutu	
1	MatrizeEredu Klasea	Oier Z, Lander V, Asier F, Ibai G	5 ordu	8 ordu	MVC patroia berregin behar izan genuen
1	Ontzia Klasea	Ibai G, Asier F	1 ordu	45 minutu	

Taula 1: 1. Sprint-eko atazen jarraipena.

2. Sprint-eko Atazak

Sprint	Ataza	Arduraduna	Denbora Planif.	Denbora Errealak	Oharrak
2	EI6: Hasierako panela aldatu, joko konfiguratzeko aukerak gehitu (Green, Blue, Red ontzien artean aukeratzeko).	Asier F, Ibai G	2 ordu	2.5 ordu	Hasierako pantailako botoiak ondo sinkronizatzea piskat kostatu zen.
2	EI7: Pixel anitzeko espaziontzia eta etsaiak implementatu.	Oier Z, Lander V, Ibai G	3 ordu	3 ordu	Irudi ezberdinak irakastea eta hitbox-ak mantentzea lortu dugu.
2	EI8: Espaziontzien tiro motak eta beren aldaketak implementatu.	Ibai G, Oier Z	2 ordu	2.5 ordu	Arma berriak sartzean tirokatu() klasea zertxobait zabaldu dugu.
2	EI9: Diseinu patroien implementazioa.	Asier F, Lander V, Ibai G	4 ordu	3.5 ordu	MVC guztiz banatu da eta patroiak ondo implementatuta gelditu dira txosteneko idatziekin batera.

Taula 2: 2. Sprint-eko atazen jarraipena.

3. Sprint-eko Atazak

Sprint	Ataza	Arduraduna	Denbora Planif.	Denbora Errealak	Oharrak
3	EI10: Java 8 ezaugarriak gehitu implementazioan (<i>Lambdas</i> , <i>forEach</i>).	Oier Z, Asier F, Ibai G	2 ordu	1.5 ordu	Begizta zaharrak aldatzean kodea asko sinplifikatu genuen, arazo handirik gabe.
3	EI11: Puntuazioaren eta bizitzen informazioa gehitu hasierako pantailan.	Ibai G, Lander V	2 ordu	2 ordu	Bista eta Ereduaren arteko sinkronizazioa (Observer bidez) ondo joan zen datuak agertzeko.
3	EI12: Etsai azkarrak eta etsaien tiroak implementatu, portaera konplexuagoarekin.	Asier F, Lander V, Oier Z, Ibai G	3 ordu	3.5 ordu	Etsaiei ausaz tirokatzea programatzeak harien arteko talkak (hitbox arazoak) ekarri zizkigun hasieran.
3	EI13: Joko modu mugagabea eta zailtasun motak implementatu.	Oier Z, Ibai G	2 ordu	2 ordu	Maila sistema gehitzean jokoaren zailtasuna orekatu behar izan genuen.

Taula 3: 3. Sprint-eko atazen jarraipena.

4.3 Proben dokumentazioa

1. Sprint-eko probak

Id	Helburua	Exekuzio baldintzak	Esperotako emaitza	Lortutako emaitza	Bukatuta
1	Espazio-matrizea hasieratu	Jokoa abiaraztean	Saretak pantailan agertzea	Ondo marraztu da	BAI
2	Ontzi mugimendua eta tiroa	Teklatuko geziak eta tarteak sakatzean	Ontzia mugitzea eta tiro egitea	Behar bezala mugitzen da	BAI
3	Etsaien mugimendua	Joko-ziklo bakoitzean (Timer)	Etsaiak behera jaitea pixkanaka	Mugimendua zuzena da	BAI
4	Hasierako panela sortu	Aplikazioa irekitzean	Pantaila nagusia bistaratzea botoiekin	UI-a ondo kargatzen da	BAI
5	MVC aplikatu	Ereduan datuak aldatzean	Bistak bere burua eguneratzea	Sinkronizazioa lortu da	BAI

2. Sprint-eko probak

Id	Helburua	Exekuzio baldintzak	Esperotako emaitza	Lortutako emaitza	Bukatuta
6	Ontzia aukeratu	Kolorea hautatzean (Gorria...)	Ontzi hori matrizean marraztea	Kolore zuzena kargatzen da	BAI
7	Pixel anitzeko ontzi/etsaiak	Ontziak eta etsaiak sortzean	Gelaxka multzo bat bezala jokatzeko	Tamaina ondo egokitu da	BAI
8	Arma/Tiro mota desberdinak	"Tiroa aldatu" tekla sakatzeko	Tiro desberdinak ateratzea	Arma ondo aldatzen da	BAI
9	Diseinu patroien frogak	Etsai edo ontzi berriak eskatzean	Klase egokitik instantzia berria bueltatzea	Zuzen lortzen dira objektuak	BAI

3. Sprint-eko probak

Id	Helburua	Exekuzio baldintzak	Esperotako emaitza	Lortutako emaitza	Bukatuta
10	Java 8 kodea testatu	Lambdak/forEach exekutatzeko	Aurreko portaera berdina mantentzea huts gabe	Akatsik gabe dabil	BAI
11	UI-ko puntuazioa eta bizitzak	Etsai bat hiltzean edo bizitza galtzean	Label-ak asinkronoki eguneratzea	UI-a ondo berritzen da	BAI
12	Etsai azkarrak eta tiroak	Joko zikloan denbora pasa ahala	Etsaiek asimetrikoki tirokatzea	Hitbox-ak eta tiroak bat datoz	BAI
13	Joko modu mugagabea	Etsai guztiak hiltzean (mugagabea)	Olatu berri bat hastea etsai indartsuagoekin	Funtzionamendu perfektua	BAI

5 Ondorioak

5.1 Hausnarketa

Proiektu honen garapenak Software Ingeniaritzan emandako kontzeptu teorikoak praktikan jartzeko aukera ezin hobe eman digu. Hasieran, MVC patroia ulertzea eta Observer bidez ondo konektatzea kostatu zitzaigun. Arazo tekniko batzuk harien(threads) kudeaketarekin sortu ziren.

Talde-lanean, Git bidezko gatazkak (merge conflicts) konpontzea oztopo bat izan zen hasierako Sprint-etan, baina denborarekin Git-en funtzionamendura ohitu eta komunikazioa hobetu genuen. Orokorrean, iritzi oso positiboa dugu. Amaierako jokoak ezin hobeto funtzionatzen duela ikustea oso aberasgarria izan da.

5.2 Etorkizuneko lanak

Jokoa hobetzeko hainbat bide irekita geratzen dira:

- Botoiak eta interfaze grafiko aurreratuagoak gehitzea, soinu eta animazio konplexuagoekin.
- Jokalari anitzeko joko modua (lokalean zein sare bidez) inplementatzea.
- Datu-base bat integratzea puntuazio altuenak gordetzeko eta mundu mailako ranking bat izateko.